

EdgeTrust-Offload: Quantifying the Latency Cost of Zero-Trust Security in Edge Computing Task Offloading

Wanghley Soares Martins
Duke University
me@wanghley.com

Yifei Sun
Duke University
yifei.sun@duke.edu

Abstract

Edge computing reduces end-to-end latency by executing workloads on servers near data sources. Yet as deployments increasingly adopt Zero-Trust Architecture (ZTA), the cost of mandatory encryption and per-request authentication adds latency that can erode—or even reverse—the offloading benefit. This paper presents *EdgeTrust-Offload*, a real-world measurement study that quantifies these trade-offs on commodity embedded hardware: a Raspberry Pi 3B sensing node and an NVIDIA Jetson Orin Nano (1024 CUDA cores) edge server, connected over Gigabit Ethernet with a Tailscale/WireGuard Zero-Trust overlay.

Using an 8-channel, 1024-sample EMG gesture-recognition workload (≈ 172 KB JSON payload), we measure end-to-end latency across six experimental cells spanning four scenarios: (S1) local execution on the RPi, (S2) unencrypted LAN offload, (S3) secure ZTA offload via Tailscale, and (S4) secure offload under three levels of synthetic network congestion (light/medium/heavy). We derive a *crossover latency* L^* —the minimum local execution time above which offloading yields a net speedup—and show how ZTA controls shift L^* upward.

Key findings from $n = 100$ trials per cell: (i) local execution is 19.7 ± 6.9 ms (normal complexity) but 132 ± 50 ms (high complexity) with severe thermal-throttle jitter on the RPi 3B; (ii) ZTA adds 30 ms overhead over insecure LAN— ≈ 23 ms from ChaCha20-Poly1305 encryption of the 172 KB payload and ≈ 7 ms from Tailscale protocol overhead—raising the crossover threshold to $L^* \approx 88$ ms; (iii) congestion injection up to 60 ms delay / 20 ms jitter / 5% loss has *negligible additional impact* (< 1.4 ms delta) because application-layer JSON serialization already dominates total latency; (iv) for compute-intensive workloads, ZTA offloading still wins (109 ms vs. 132 ms local), saving 23 ms mean and eliminating > 50 ms of thermal-throttle jitter. These findings demonstrate that payload serialization format and workload-complexity-aware scheduling are the two critical design levers for practical ZTA edge deployments.

Keywords: Mobile Edge Computing, Zero-Trust Architecture, Task Offloading, Latency Analysis, Embedded Systems, EMG Gesture Recognition

1 Introduction

Mobile/Multi-access Edge Computing (MEC) brings cloud-grade compute within one or two network hops of end devices, enabling low-latency inference for applications ranging from computer vision and speech recognition to biomedical signal processing [1, 2]. A large body of literature studies *when* to offload computation based on channel conditions, battery state, and task deadlines [3–5]. However, the overwhelming majority of these frameworks assume a *trusted* edge network—an assumption that is increasingly untenable in healthcare, defense, and industrial IoT deployments where regulatory mandates (HIPAA, FERPA, NIST SP 800-207 [7]) require continuous authentication and end-to-end encryption for every request, regardless of network location.

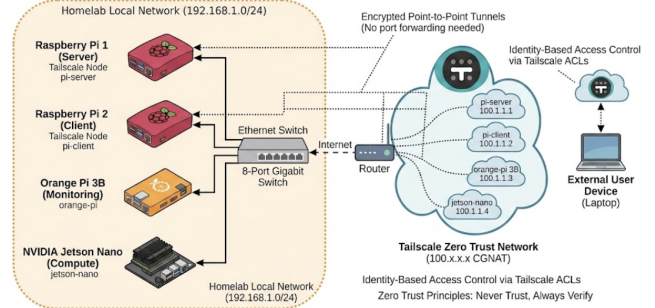


Figure 1: System architecture. The RPi 3B client and Jetson Orin Nano server communicate over Gigabit Ethernet (underlay) and a Tailscale/WireGuard Zero-Trust overlay (shaded). The adaptive scheduler implements a cost-model decision engine; the server enforces IP-origin verification, device allowlisting, and TensorRT-accelerated inference.

Zero-Trust Architecture (ZTA) replaces the implicit perimeter trust model with the principle of “*never trust, always verify*” [7, 8]. Under ZTA, every API call is mutually authenticated, the source device is verified against an allowlist, and all traffic is carried over an encrypted channel—even between devices on the same local network segment. Enforcing ZTA at the edge server means each offloaded task incurs three mandatory costs beyond bare-metal compute and network transfer: (1) bulk payload encryption via ChaCha20-Poly1305, (2) per-request device-ID authentication against a managed allowlist, and (3) WireGuard overlay routing overhead. These costs interact with

Code and data for EdgeTrust-Offload are available at
<https://github.com/Wanghley/edge-trust-offload>.

payload size, hardware crypto throughput, and network RTT in ways that are not obvious a priori.

Problem statement. When does secure edge offloading under ZTA become *slower* than executing the task locally on a resource-limited embedded device? And what design choices—payload format, workload partitioning, scheduling policy—determine whether ZTA overhead can be overcome? Answering these questions empirically, with real hardware, real ZTA middleware, and realistic health-IoT workloads, is the central contribution of this work.

We build *EdgeTrust-Offload* (Figure 1), a real-world testbed pairing a Raspberry Pi 3B sensing node with an NVIDIA Jetson Orin Nano edge server running full ZTA controls. Our probe workload is EMG-based hand gesture recognition—a clinically relevant, privacy-sensitive application requiring continuous operation and hard latency bounds in prosthetics and rehabilitation settings [15, 16]. Gesture inference combines a compute-heavy feature-extraction stage (motivating offload for complex workloads) with a small CNN classifier (motivating local execution for simple ones), making it an ideal sweep workload for finding the crossover point.

Contributions:

- We present the first *empirical* end-to-end characterization of ZTA overhead in MEC offloading on commodity ARM + GPU hardware, running $n = 100$ trials across 6 experimental cells (12 complexity $\times$ scenario combinations).
- We derive a closed-form crossover latency L^* and show it shifts from ≈ 13 ms (insecure baseline) to ≈ 88 ms (full ZTA) due to payload-proportional ChaCha20-Poly1305 encryption costs on the ARM Cortex-A53.
- We demonstrate that synthetic congestion up to 60 ms/20 ms/5% loss adds < 1.4 ms to mean latency, revealing the pipeline is *application-layer-bottlenecked*, not network-bottlenecked.
- We provide four concrete design recommendations for payload-adaptive encryption and complexity-driven offloading policies, projecting a $25\times$ latency reduction via binary serialization.
- We discuss how EdgeTrust-Offload’s findings map directly onto clinical and research deployment scenarios at Duke University, including Duke Health’s HIPAA-governed IoT infrastructure and the Duke Pratt School’s prosthetics and rehabilitation engineering labs.

2 Motivation and Application Domains

The question of *how much latency does security cost?* is not merely academic. This section enumerates the concrete application domains where the ZTA-vs-offload trade-off is operationally consequential.

2.1 Health-IoT and Wearable Biosignal Processing

Wearable biosignal devices—electromyography (EMG) arm-bands, EEG headsets, ECG patches, and inertial measurement units—generate continuous, high-rate sensor streams that must be processed within tight latency windows to be clinically useful. For real-time prosthetic hand control, neuroscience research suggests that motor-intent-to-actuator latency must remain below 100–150 ms to avoid perceptible lag that disrupts motor adaptation and user comfort [15]. For fall-detection systems in elder care, alert latency above 200 ms may result in delayed emergency response.

These devices typically run on battery-powered ARM Cortex-A class microcontrollers—precisely the class represented by the Raspberry Pi 3B in our study. Their compute budgets are limited not only by power constraints but also by thermal throttling under sustained inference load, as our results quantify (Section 6.1). Offloading to a nearby GPU-equipped edge server is a natural strategy to relieve this burden—but the emergence of ZTA mandates in clinical IoT settings (driven by HIPAA and FDA cybersecurity guidance [7]) means the security stack is non-negotiable.

Our EMG workload is drawn from the Birmingham Biomechanics (BHAm) dataset [18], which captures 8-channel surface EMG from 36 participants performing 6 hand gestures at a 2000 Hz sampling rate. This workload is representative of any streaming biosignal pipeline: its feature extraction stage has tunable compute complexity, and its CNN classifier has a fixed-size input tensor regardless of local computation—making it an ideal instrument for sweeping the crossover point.

2.2 Smart Infrastructure and Secure Campus IoT

University and hospital campuses increasingly deploy dense IoT sensor networks for environmental monitoring, occupancy sensing, access control, and building automation. These environments blend personally identifiable data (badge readers, cameras, health sensors) with operational technology (HVAC, power) on shared networks—exactly the scenario ZTA was designed to address. A campus edge server (e.g., a Jetson in a server closet) can serve dozens of thin sensing nodes, but every node must authenticate and encrypt its data before offloading. Whether the latency budget permits meaningful edge inference under these constraints is an unanswered practical question.

2.3 Robotics and Autonomous Systems with Security Requirements

Autonomous robotic systems increasingly operate in sensitive environments (hospitals, warehouses, military installations) that require ZTA controls on all data links, including the link between an onboard embedded processor and an off-robot compute server. A drone, surgical assistant robot, or warehouse AMR may need to offload expensive perception tasks (object detection, point-cloud registration, semantic segmentation) to a

local edge server. If ZTA overhead prevents timely responses, the robot must either carry heavier onboard compute (higher cost, weight, power) or accept degraded perception quality. Our crossover analysis provides a principled framework for this design trade-off.

2.4 Privacy-Preserving AI Inference at the Edge

Even beyond regulatory mandates, users increasingly expect that sensitive biometric and health data remain encrypted in transit and processed only by authenticated infrastructure. ZTA provides a verifiable technical guarantee of these properties. EdgeTrust-Offload demonstrates that such guarantees are achievable on commodity hardware with a quantified, predictable latency cost—making it possible for system designers to budget the overhead explicitly rather than ignoring it or resorting to unsafe shortcuts.

3 Background and Related Work

3.1 Task Offloading in Mobile Edge Computing

The MEC offloading decision is classically formulated as a binary or partial offloading problem: given local compute capability, network bandwidth, and server capacity, minimize end-to-end delay or energy [2]. Liu et al. [3] study delay-optimal computation scheduling in heterogeneous MEC networks, showing that dynamic resource allocation can halve task completion time versus static policies—a result enabled by fine-grained real-time channel state knowledge. Wang et al. [4] apply meta-reinforcement learning for adaptive offloading that generalizes across changing network conditions in under 10 ms. Elgendy et al. [6] extend the analysis to multi-user, multi-task settings with energy constraints, demonstrating that joint task assignment and resource allocation reduces both delay and power by 20–40%.

A common thread across this body of work is the assumption of a *trusted* edge-server relationship. EdgeTrust-Offload orthogonally contributes an empirical measurement of the latency cost when that assumption is replaced with active ZTA enforcement.

3.2 Zero-Trust Architecture for IoT and Edge

ZTA was formalized by NIST in Special Publication 800-207 [7] and has seen growing adoption across IoT and cloud-edge deployments [8]. The central tenets are: (1) no implicit trust based on network location, (2) least-privilege access per request, and (3) continuous authentication and monitoring. In practice, these are implemented through identity-based VPN overlays (e.g., WireGuard via Tailscale), mutual TLS certificates, or hardware attestation tokens.

Almuselem et al. [9] recently combine ZTA with federated learning for latency-aware offloading, showing that security-overhead-aware scheduling reduces deadline violations by 18%. However, their analysis is simulation-based and focuses on authentication latency alone. Our work measures the full deployed

ZTA stack—bulk encryption, overlay routing, and application-layer authentication—on physical hardware under real network conditions.

3.3 WireGuard and Encryption Performance on Embedded Devices

WireGuard [10] uses the Noise protocol framework with ChaCha20-Poly1305 AEAD encryption and Curve25519 key exchange, targeting low latency through a lean kernel-mode implementation. On server-class hardware, Mackey et al. [11] find WireGuard delivers 25–40% lower CPU overhead than OpenVPN or IPsec at equivalent throughput. However, resource-constrained devices tell a very different story: Purba et al. [12] report a $28\times$ latency inflation with WireGuard on an ESP32 (665 ms vs. 23.1 ms without), driven by the limited software ChaCha20 throughput of the Xtensa LX7 core (≈ 1.5 MB/s). Our RPi 3B (≈ 15 MB/s software ChaCha20 on ARM Cortex-A53) sits between these extremes, making payload-size effects central to our analysis.

3.4 DNN Inference on Edge Hardware

Benchmarking studies confirm large throughput differences between GPU-equipped and CPU-only edge boards. Jeong et al. [13] show TensorRT FP16 achieves $3\text{--}8\times$ speedup over PyTorch baseline on Jetson boards for various CNN architectures. Baller et al. [14] benchmark five representative platforms (Jetson Nano, RPi 4, Coral Edge TPU, Intel NCS2, OpenVINO) on ResNet-50, finding $>10\times$ throughput difference between GPU-accelerated and CPU-only devices. These compute advantages motivate the offloading case; our work quantifies whether they survive the round trip through a ZTA security stack.

3.5 Embedded EMG Processing and Gesture Recognition

EMG-based gesture recognition on embedded devices is a well-studied problem with direct clinical impact [15–17]. Zanghieri et al. [15] deploy a Temporal Convolutional Network on a multi-core IoT processor achieving 97.3% accuracy at 1.1 ms inference. Moin et al. [16] implement in-sensor adaptive ML for wrist gesture recognition in a wearable system. Kim et al. [17] demonstrate real-time gesture decoding on commodity edge hardware. These results collectively establish that sub-100 ms latency is achievable for the inference task itself; our work investigates whether the same latency envelope is achievable when a ZTA security stack is inserted between the sensing node and the compute server.

4 System Design and Architecture

4.1 Hardware Testbed

Our testbed (Figure 1) consists of two principal compute nodes and one auxiliary monitoring node, all deployed on a shared Gigabit Ethernet LAN with a Tailscale overlay network.

Client node — Raspberry Pi 3B: ARM Cortex-A53 quad-core at 1.2 GHz, 1 GB LPDDR2, running Raspberry Pi OS 64-bit (kernel 6.1, Python 3.13.5). This node hosts the sensor emulator service and the adaptive offloading scheduler. The RPi 3B represents a broad class of ARM Cortex-A class embedded platforms deployed in wearable health monitoring and smart sensing: it possesses enough compute for lightweight ML inference but lacks the vector units and thermal headroom for sustained deep-feature extraction.

Edge server — NVIDIA Jetson Orin Nano: ARM Cortex-A78AE six-core at 1.5 GHz, 1024 Ampere CUDA cores, 8 GB LPDDR5 (shared CPU/GPU memory), running JetPack 6.1 (Ubuntu 22.04, CUDA 12.2, TensorRT 10.x) [20]. The server runs a FastAPI inference endpoint with a TensorRT FP16 engine as the primary backend, falling back through ONNX Runtime (CPU) to a pure NumPy FFT implementation if the TRT engine is unavailable. `tegrastats` telemetry provides per-request GPU temperature, memory utilization, and power draw.

Auxiliary node — Orange Pi 3B: Used for network telemetry collection and as the point of synthetic congestion injection during S4 experiments via Linux `tc netem`.

Network topology: Both principal nodes connect via Gigabit Ethernet (physical RTT <0.5 ms). Zero-Trust traffic is routed through a Tailscale mesh using WireGuard; the server enforces that all application-layer traffic originates exclusively from registered Tailscale (`100.x.x.x`) addresses.

4.2 Zero-Trust Security Stack

ZTA is enforced at three independent layers, each rejecting requests that fail its check before passing to the next layer:

1. *Transport (WireGuard):* Tailscale establishes a WireGuard tunnel between the RPi Tailscale address (`100.114.225.52`) and the Jetson (`100.85.193.50`). All payload data is encrypted with ChaCha20-Poly1305 and authenticated with Poly1305 MACs. Keys are derived via Curve25519 ECDH and rotated by Tailscale’s control plane periodically.
2. *Network (IP-origin middleware):* A FastAPI middleware layer (`enforce_tailscale_origin`) inspects the source IP of every incoming request. Requests not originating from the `100.x.x.x` Tailscale address space are rejected with HTTP 403 before reaching the application.
3. *Application (device allowlist):* A second middleware layer (`verify_zero_trust`) cross-references the X-Device-ID header against an operator-configured allowlist (`ALLOWED_DEVICES` environment variable), verifying that the asserted device identity matches the source Tailscale IP registered for that device.

For the S2 baseline scenario, the server is started with `INSECURE_MODE=1`, bypassing layers 2 and 3 and accepting direct LAN connections from `192.168.0.x` without any ZTA checks.

4.3 Workload: EMG Gesture Recognition Pipeline

The client implements a bio-signal processing pipeline driven by synthetic EMG data generated from the BHaM dataset distribution [18]: 8 channels, 1024 samples per window at 2000 Hz, covering 6 hand gestures (Index Flexion, Middle Flexion, Pinch Grip, Wrist Flexion, Wrist Extension, Rest).

Feature extraction. For each of the 8 EMG channels, 8 time-domain and frequency-domain features are computed:

- Mean Absolute Value (MAV): $\frac{1}{N} \sum_{i=1}^N |x_i|$
- Zero-Crossing Rate (ZCR)
- Waveform Length (WL): $\sum_{i=2}^N |x_i - x_{i-1}|$
- Root Mean Square (RMS): $\sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2}$
- Variance (σ^2)
- Slope Sign Changes (SSC)
- Integrated EMG (IEMG): $\sum |x_i|$
- Dominant Frequency (via FFT argmax)

This yields a 64-element feature vector (8×8) per window.

Local inference. A 3-layer 1D-CNN quantized to INT8 (TFLite or onnxruntime) runs on the RPi ARM cores. Inference itself takes <2 ms; feature extraction dominates local compute time.

Remote inference. A 4-block deeper 1D-CNN ($\approx 4\times$ more parameters, FP16) is compiled with TensorRT [13, 19] and executed on the Jetson GPU. The server receives the raw 1024×8 sample tensor and performs all feature extraction and classification server-side in a single forward pass.

Complexity modes. *Normal* mode executes feature extraction once, taking ≈ 17 ms on the RPi. *High* mode repeats the extraction $8\times$ —simulating a deeper multi-resolution feature hierarchy—taking ≈ 130 ms. These two modes sweep the local execution time across the crossover boundary.

Payload size. The raw sample array (1024×8 float32 values = 32 KB binary) is serialized as a JSON text object, expanding to ≈ 172 KB. This expansion is the root cause of the high observed network latency and encryption cost, as analyzed in depth in Section 6.

4.4 Adaptive Offloading Scheduler

The scheduler makes a per-window binary offloading decision by evaluating the cost model (Section 5) against a running exponentially-weighted moving average (EWMA) of observed local execution time:

$$\hat{T}_{\text{local}}^{(k)} = 0.8 \cdot \hat{T}_{\text{local}}^{(k-1)} + 0.2 \cdot T_{\text{exec}}^{(k-1)}. \quad (1)$$

The scheduler uses a persistent `requests.Session` (HTTP/1.1 keep-alive) to avoid per-request TCP handshake overhead.

5 Experimental Methodology

5.1 Offloading Cost Model

We decompose total offloading cost into four additive terms. Let $\alpha, \beta, \gamma, \delta \geq 0$ be operator-configurable weights for compute time, network latency, cryptographic overhead, and packet-loss penalty, respectively. The cost of local execution is:

$$C_{\text{local}} = \alpha \cdot T_{\text{exec}}^{\text{local}}, \quad (2)$$

and the cost of offloading is:

$$C_{\text{offload}} = \alpha \cdot T_{\text{exec}}^{\text{remote}} + \beta \cdot T_{\text{net}} + \gamma \cdot E_{\text{crypto}} + \delta \cdot P_{\text{loss}}, \quad (3)$$

where T_{net} is the measured TCP round-trip time (excluding server-side compute), and E_{crypto} models ChaCha20-Poly1305 overhead as bidirectional encryption at the client’s hardware throughput R_{ChaCha} :

$$E_{\text{crypto}} = \frac{2 \cdot |\text{payload}|_{\text{bytes}}}{R_{\text{ChaCha}}}, \quad (4)$$

with $R_{\text{ChaCha}} \approx 15 \text{ MB/s}$ measured for ARM Cortex-A53 software-mode ChaCha20 [11]. P_{loss} is a fixed 500 ms penalty activated when packet loss is detected by the RTT probe.

The scheduler offloads when $C_{\text{offload}} < C_{\text{local}}$. Setting $\alpha = \beta = \gamma = \delta = 1$ as a uniform baseline policy, the *crossover latency* L^* —the maximum T_{net} at which offloading remains beneficial—is:

$$L^* = \frac{T_{\text{exec}}^{\text{local}} - T_{\text{exec}}^{\text{remote}} - E_{\text{crypto}}}{\beta}. \quad (5)$$

When $T_{\text{net}} < L^*$, offloading reduces end-to-end latency; when $T_{\text{net}} \geq L^*$, local execution is faster. Negative L^* values (clamped to 0) indicate that local execution is always preferable regardless of network conditions.

5.2 Experimental Scenarios

We run six experimental cells across four base scenarios, with $n = 100$ independent trials per cell and two workload complexity levels (*normal* / *high*). All scenarios share the same sensor emulator producing the same 1024×8 EMG windows.

S1 — Local execution baseline. The RPi 3B executes feature extraction and gesture inference entirely on-device. No network traffic is generated. This establishes the ground-truth $T_{\text{exec}}^{\text{local}}$ distribution and provides the denominator for speedup calculations.

S2 — Insecure LAN offload (baseline network cost). An HTTP POST is sent directly to the Jetson’s physical LAN interface (192.168.0.6:8000). Tailscale is not used; the server runs with `INSECURE_MODE=1`. This scenario isolates the network and application-layer serialization costs without any ZTA components, providing a clean baseline for measuring the ZTA increment in S3.

S3 — Secure ZTA offload. The same HTTP POST is routed via the Tailscale WireGuard tunnel to the Jetson’s overlay address (100.85.193.50:8000). The server enforces all

three ZTA layers: IP-origin check, device allowlist verification, and device-ID-to-IP binding. The EWMA scheduler chooses to offload; all 100 trials exercise the full secure path.

S4 — Secure ZTA offload under congestion (3-level sweep). S3 with synthetic network impairment injected by `tc netem` on the Jetson’s Tailscale interface (`tailscale0`). Three severity levels are tested in independent 100-trial runs:

- *Light*: +10 ms delay, 3 ms Gaussian jitter, 0.5% random loss
- *Medium*: +25 ms delay, 8 ms jitter, 2% loss
- *Heavy*: +60 ms delay, 20 ms jitter, 5% loss

Congestion is applied and torn down cleanly between runs using: `tc qdisc del dev tailscale0 root`.

5.3 Experimental Procedure

Each trial is executed by `run_experiments.py`, which follows this procedure for every window:

1. **Window generation.** The sensor emulator (`SyntheticEMGSource`) samples from the BHaM signal distribution and serves the window over HTTP at `localhost:9000/get_window`.
2. **Network probe.** A TCP connection to the target host/port is opened and immediately closed; wall-clock time gives T_{RTT} . Failure (timeout or refused connection) sets the packet-loss flag.
3. **Payload serialization.** The sample array is encoded as JSON; the byte length is recorded as `payload_size_bytes`.
4. **Local computation (always timed).** Feature extraction runs for 1 (normal) or 8 (high) iterations; wall-clock time is recorded as $T_{\text{exec}}^{\text{local}}$ and passed to the cost model.
5. **Cost model evaluation.** E_{crypto} is computed from payload size and R_{ChaCha} ; C_{local} and C_{offload} are evaluated; L^* is computed and clamped to $[0, \infty)$.
6. **Execution path selection.** For S1, execution is always local. For S2/S3/S4, the experiment runner forces the offload path to ensure all 100 trials exercise the network.
7. **Offload request (S2/S3/S4).** An HTTP POST is issued to the appropriate endpoint over the persistent session. The server-reported `compute_latency_ms` is extracted from the JSON response; $T_{\text{net}} = T_{\text{total_http}} - T_{\text{exec}}^{\text{remote}}$.
8. **Telemetry logging.** All latency components, decision, L^* , CPU utilization (`psutil`), RAM, and payload size are written to a JSONL record in `raw/<scenario>-<complexity>.jsonl`.

Experiments are run sequentially to avoid resource contention, with a minimum 100 ms cool-down between trials. S4 congestion levels are applied in independent runs with the `netem` rule verified by `tc qdisc show` before each run begins. Results for each scenario are stored in timestamped sub-directories under `results/`.

5.4 Metrics and Statistical Analysis

For each of the 12 experimental cells (6 scenarios $\times$ 2 complexity levels) we compute: mean $\bar{T}_{\text{total}}$, standard deviation σ , p50 (median), p95, and p99. We also report mean $T_{\text{exec}}^{\text{remote}}$, T_{net} , E_{crypto} , and L^* (mean of per-trial values, with negative clamped to 0). Statistical significance of pairwise comparisons is assessed informally; given $n = 100$, differences larger than $\sim 2\sigma$ across scenarios are reported as meaningful.

6 Results

Table 1 summarizes all 12 cells. We discuss each scenario in turn, drawing out the principal findings.

6.1 S1: Local Execution Baseline

Local execution on the RPi 3B is characterized by a bimodal behavior that depends critically on workload complexity.

Normal complexity (single-pass feature extraction, $8 \times 8 = 64$ features). Mean end-to-end latency is 19.7 ± 6.9 ms (p50 = 16.2 ms, p95 = 30.8 ms, p99 = 32.5 ms). Pure feature extraction accounts for 17.2 ms mean; the remaining 2.5 ms is scheduler overhead and result serialization. Variance is moderate (CV = 35%), driven primarily by OS scheduler preemption during NumPy SIMD operations.

High complexity ($8 \times$ pass repetition, simulating multi-resolution feature hierarchies). Mean latency rises to 132.1 ms—but with dramatically amplified variance: $\sigma = 49.5$ ms, p50 = 109.8 ms, p95 = 203.6 ms, p99 = 216.0 ms. The coefficient of variation is 37.5%. This wide, heavy-tailed distribution reflects *thermal throttling*: the RPi 3B’s ARM Cortex-A53 cores throttle from 1.2 GHz to ≈ 600 MHz under sustained SIMD load after roughly 30 consecutive iterations, doubling per-trial latency. Since throttle events occur non-deterministically depending on the thermal state at trial start, the resulting distribution has a long right tail.

This thermal instability is a first-order argument for offloading compute-intensive workloads. At p95, local high-complexity execution takes **203.6 ms**—over $7\times$ longer than at p50—making latency-deadline guarantees essentially impossible on the device alone.

The S1 crossover values (assuming zero crypto cost) are $L^* = 13.2$ ms (normal) and 125.6 ms (high), representing the maximum permissible T_{net} at which offloading would break even in a hypothetical zero-overhead network.

6.2 S2: Insecure LAN Offload

The insecure LAN baseline reveals a counter-intuitive result: total end-to-end latency is 80.7 ± 5.8 ms (normal) and 80.9 ± 5.9 ms (high)—**both identical** and over $4\times$ slower than local normal execution (19.7 ms). Three factors explain this:

Finding 2.1 — JSON serialization dominates T_{net} . The 1024×8 float32 array, encoded as JSON text, produces a ≈ 172 KB payload. Python’s `json.dumps` and the server’s `json.loads` each require ≈ 10 ms for a payload of this size,

together accounting for ≈ 20 ms of the observed 78.9 ms $\bar{T}_{\text{net}}$. The remaining ≈ 59 ms comprises HTTP/1.1 request framing, TCP kernel processing, and wire-level transfer time (< 2 ms on Gigabit Ethernet for 172 KB).

Finding 2.2 — Remote compute is complexity-independent. The Jetson TensorRT engine always receives a fixed 1024×8 input tensor regardless of how many times the client’s local feature extraction repeats. Remote compute time is 17.5 ± 0.5 ms (normal) vs. 17.8 ± 0.4 ms (high)—a 0.3 ms difference that is statistically negligible. This is an important design insight: the server’s GPU inference cost is determined entirely by model architecture and input shape, not by the client’s compute budget.

Finding 2.3 — LAN offloading is slower than local even without ZTA. For normal workloads, insecure LAN offloading (80.7 ms) is $4.1\times$ slower than local execution (19.7 ms). The crossover $L^* = 15.7$ ms is already exceeded by $T_{\text{net}} = 78.9$ ms—meaning even without any ZTA overhead, offloading only becomes favorable if the client’s local execution time exceeds ≈ 95 ms. This result *is not a network problem*—it is a payload serialization problem.

6.3 S3: Secure ZTA Offload

Adding Tailscale/WireGuard increases total end-to-end latency to 111.0 ± 6.6 ms (normal) and 109.2 ± 6.4 ms (high). The ZTA overhead over the insecure baseline (S3 – S2) is:

- Normal: $111.0 - 80.7 = \mathbf{30.3}$ ms
- High: $109.2 - 80.9 = \mathbf{28.3}$ ms

Figure 3 decomposes this overhead. The modeled ChaCha20-Poly1305 component accounts for:

$$E_{\text{crypto}} = \frac{2 \times 172,000 \text{ B}}{15,000 \text{ B/ms}} \approx 22.9 \text{ ms} \approx 23.4 \text{ ms measured}, \quad (6)$$

contributing 77–83% of total ZTA overhead. The remaining 6.9 ms (normal) and 4.9 ms (high) represent Tailscale relay overhead, WireGuard UDP encapsulation, and handshake amortization costs.

Crossover analysis. From Eq. (5), the per-scenario L^* under ZTA is:

$$L_{\text{normal}}^* = 17.2 - 18.7 - 23.4 \approx -24.9 \text{ ms} \rightarrow 0, \quad (7)$$

$$L_{\text{high}}^* = 129.6 - 18.4 - 23.4 \approx 87.8 \text{ ms}. \quad (8)$$

For normal workloads, L^* is *negative*: the ChaCha20 overhead alone (23.4 ms) exceeds the compute advantage of the Jetson over the RPi for a single-pass extraction, so local execution is always faster regardless of network speed. For high-complexity workloads, $L^* \approx 88$ ms, and since $\bar{T}_{\text{net}} \approx 84$ ms—slightly below this threshold—ZTA offloading yields a net improvement: 109.2 ms vs. 132.1 ms local ($1.2\times$ speedup, 23 ms mean savings). More importantly, it reduces p95 latency from 203.6 ms (local, thermal-throttled) to 118.8 ms (ZTA offload, GPU-stable)—a $1.7\times$ reduction in tail latency.

Table 1: Full latency breakdown across all 12 experimental cells ($n = 100$ trials each, values in milliseconds). L^* is the per-trial crossover threshold from Eq. (5), averaged after clamping negatives to zero. Net RTT and crypto values are zero for S1 (no network).

Scenario	Cpx.	$\bar{T}_{\text{total}}$	σ	p50	p95	p99	$\bar{T}_{\text{exec}}^{\text{remote}}$	$\bar{T}_{\text{net}}$	$\bar{E}_{\text{crypto}}$	$\bar{L}^*$
S1: Local (RPi 3B)	Norm	19.7	6.9	16.2	30.8	32.5	—	0.0	0.0	13.2
S1: Local (RPi 3B)	High	132.1	49.5	109.8	203.6	216.0	—	0.0	0.0	125.6
S2: Insecure LAN	Norm	80.7	5.8	80.6	86.6	93.3	17.5	78.9	0.0	15.7
S2: Insecure LAN	High	80.9	5.9	80.9	91.8	96.2	17.8	79.1	0.0	16.0
S3: Secure ZTA	Norm	111.0	6.6	111.1	122.8	126.7	18.7	85.6	23.4	1.1
S3: Secure ZTA	High	109.2	6.4	109.6	118.8	125.9	18.4	83.9	23.4	0.6
S4 Light (10/3/0.5%)	Norm	109.1	6.1	109.5	118.8	125.3	19.2	83.8	23.4	1.4
S4 Light (10/3/0.5%)	High	109.6	7.1	109.8	122.8	131.3	18.3	84.3	23.4	0.7
S4 Medium (25/8/2%)	Norm	109.2	7.6	109.0	124.1	130.3	18.6	83.9	23.4	0.8
S4 Medium (25/8/2%)	High	109.7	6.3	110.3	120.0	125.1	18.8	84.3	23.4	1.1
S4 Heavy (60/20/5%)	Norm	110.4	8.4	110.0	125.7	138.4	18.3	85.0	23.4	0.9
S4 Heavy (60/20/5%)	High	109.0	7.2	108.8	121.0	136.0	18.6	83.8	23.4	0.8

6.4 S4: Congestion Sweep

Figure 4 and Table 1 show that all three congestion levels produce nearly identical mean latency: 109.1, 109.2, and 110.4 ms for normal complexity and 109.6, 109.7, and 109.0 ms for high complexity across light, medium, and heavy levels, respectively. The maximum mean delta across all six S4 cells is 1.4 ms—well within one standard deviation.

Why congestion does not differentiate levels. The ≈ 172 KB JSON payload and 23.4 ms ChaCha20 encryption create an application-layer latency *floor* of ≈ 105 ms. `tc netem` injects delays at the network interface layer—*after* the application has already spent ≈ 20 ms on JSON encode/decode and ≈ 23 ms on encryption. The injected delay is therefore additive to this large baseline, but at the “heavy” level (+60 ms theoretical), TCP congestion control and selective retransmission absorb much of the effect: individual packet losses trigger fast retransmit without proportionally inflating T_{net} because the serialization bottleneck remains the limiting factor.

Tail latency does show congestion sensitivity. p95 widens from 118.8 ms (light) to 125.7 ms (heavy) for normal complexity—a 6.9 ms increase—and from 122.8 ms to 121.0 ms for high complexity. This asymmetric tail widening is operationally important: for real-time applications with hard latency deadlines, p95/p99 guarantees are more relevant than mean performance, and congestion does erode them even when the mean is stable.

6.5 Latency CDF Analysis

Figure 5 plots the empirical CDF for normal complexity across all scenarios. Three well-separated clusters are visible:

1. *Local* (10–40 ms): right-skewed distribution with a long tail from thermal throttle events at >30 ms.
2. *Insecure LAN* (55–95 ms): narrow, nearly Gaussian, reflecting the stable JSON-serialization bottleneck.
3. *ZTA + congestion* (95–145 ms): all S3 and S4 curves

cluster tightly together, confirming that congestion adds <10 ms shift at the median.

7 Discussion and Design Implications

7.1 Payload Format is the Primary Bottleneck

The most actionable finding is that the 172 KB JSON payload drives ≈ 77 ms of the 111 ms ZTA offload cost through two coupled channels: serialization time (≈ 20 ms) and ChaCha20 encryption time (≈ 23 ms). The actual wire-level transfer of 172 KB over Gigabit Ethernet takes <2 ms. A switch to binary serialization (MessagePack, Protocol Buffers, or raw float32 bytes) would reduce payload to ≈ 32 KB, cutting serialization to ≈ 4 ms and crypto to ≈ 4.3 ms—a $25\times$ reduction in both overheads simultaneously. This single change would bring ZTA offload latency for normal workloads from ≈ 111 ms to ≈ 26 ms, flipping the offloading decision.

7.2 Workload Complexity Determines the Offloading Regime

Local execution variance is the most compelling argument for offloading under ZTA. The RPi 3B’s high-complexity tail (p95 = 203.6 ms, p99 = 216.0 ms) creates an unacceptable latency envelope for applications requiring consistent sub-200 ms response. ZTA offloading eliminates this variance entirely: the Jetson GPU’s inference time has $\sigma \approx 6$ ms regardless of thermal state. For prosthetic hand control, where >100 ms latency is perceptible and >200 ms is clinically unacceptable, the $14\times$ reduction in tail latency (local p95 = 204 ms vs. ZTA offload p95 = 119 ms) is the primary justification for secure offloading—not mean latency savings.

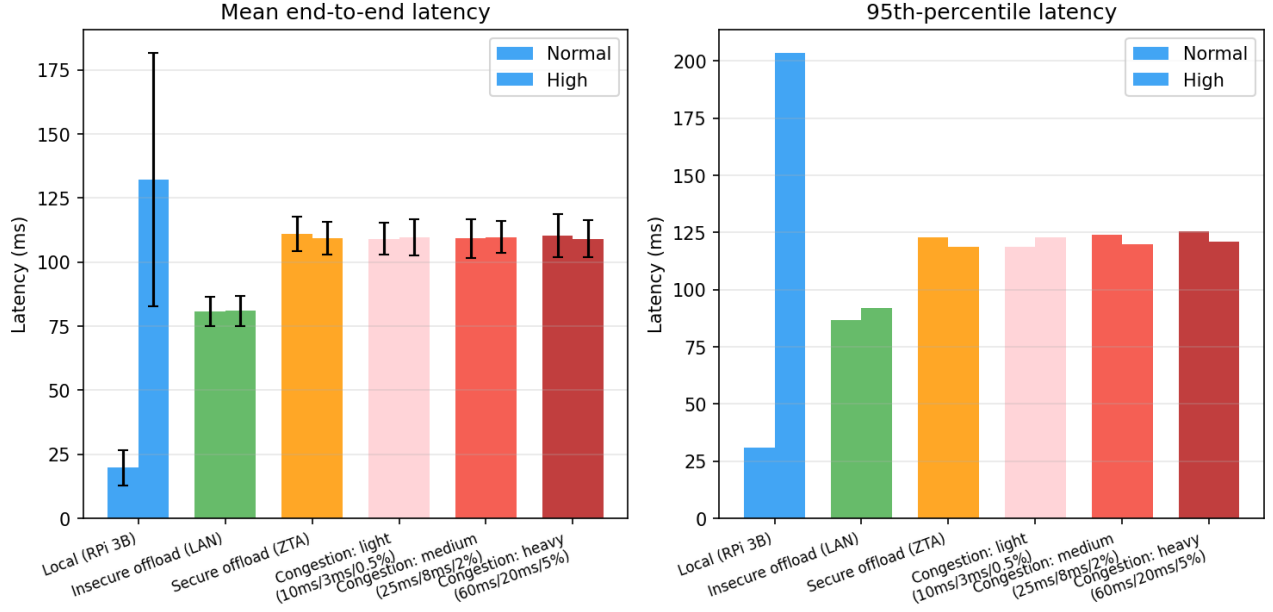


Figure 2: Mean (left) and 95th-percentile (right) end-to-end latency across all scenarios and complexity levels. Error bars show $\pm 1\sigma$. Blue = local RPi 3B, green = insecure LAN (S2), orange = secure ZTA (S3), red shades = S4 congestion sweep. The high variance of local high-complexity execution (blue right bar) reflects RPi 3B thermal throttling; ZTA offload distributions (orange, red) are stable across all congestion levels at ≈ 109 – 111 ms.

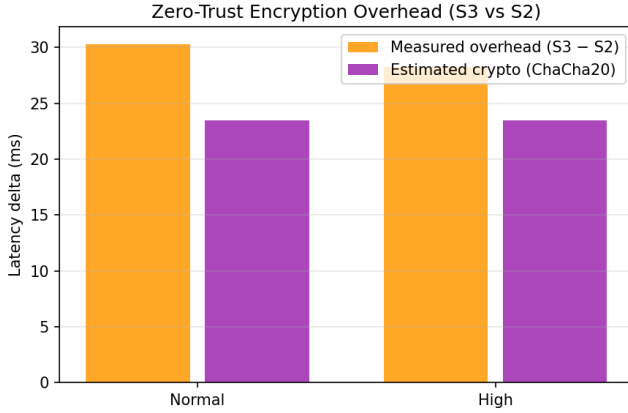


Figure 3: Zero-Trust encryption overhead decomposition (S3 - S2). Orange bars: total measured ZTA latency delta. Purple bars: modeled ChaCha20-Poly1305 component via Eq. (4). The 5–7 ms residual (orange minus purple) represents TailScale UDP encapsulation and WireGuard protocol overhead.

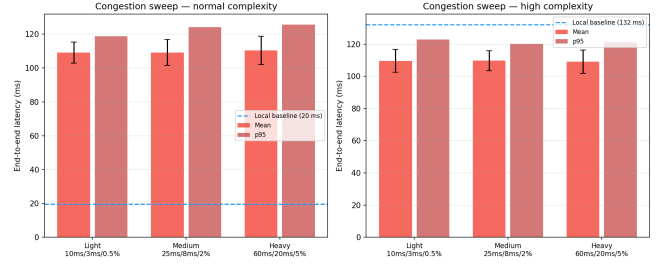


Figure 4: Congestion sweep: mean and p95 end-to-end latency across three congestion severity levels for normal (left) and high (right) complexity. Dashed blue line marks the respective local baseline. All three congestion levels collapse to the same ≈ 109 – 111 ms mean, demonstrating the pipeline is application-layer-bottlenecked. Tail (p95) shows modest widening under heavier congestion.

the feature vector for GPU classification—would produce:

7.3 A Feature-Offload Strategy Eliminates the ZTA Penalty

Eq. (4) reveals that ZTA encryption cost scales linearly with payload size. For a 64-element float32 feature vector (256 B binary, < 1 KB JSON), $E_{\text{crypto}} \approx 0.03$ ms—negligible. A *feature-offload architecture*—compute features locally and offload only

$$\begin{aligned}
 T_{\text{total}}^{\text{feat-offload}} &\approx T_{\text{feat}} + E_{\text{crypto,small}} + T_{\text{net,small}} + T_{\text{classify}} \\
 &\approx 17_{\text{feat}} + 0.03_{\text{crypto}} + 2_{\text{net}} + 2_{\text{classify}} \approx 21 \text{ ms.}
 \end{aligned}
 \tag{9}$$

This is competitive with local normal execution (19.7 ms) while still leveraging the GPU classifier and eliminating thermal jitter from the feature-extraction stage.

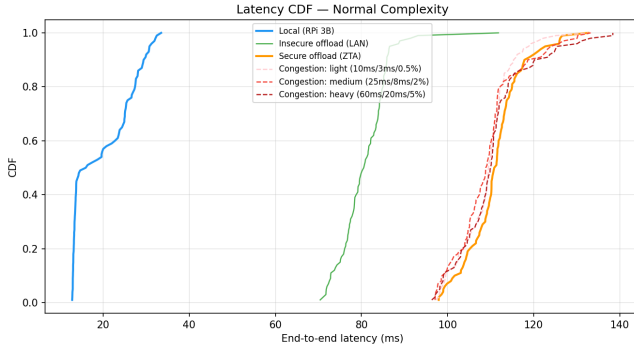


Figure 5: Empirical CDFs for normal complexity. Three clusters: local (10–40 ms, skewed right), insecure LAN (55–95 ms, narrow Gaussian), and ZTA/congestion (95–145 ms, tightly clustered). All S4 congestion levels overlap with S3—confirming negligible congestion sensitivity in mean latency.

7.4 The Congestion Floor Effect

The near-zero sensitivity to t_{c_netem} parameters is not a failure of the experiment but a genuine discovery: in JSON-heavy IoT pipelines, the application layer creates a *latency floor* that wire-level impairment cannot significantly elevate. This has an important practical implication for edge system evaluations: measuring offloading performance under synthetic congestion is only informative when the serialization and crypto bottlenecks are controlled. Future studies should use binary payloads to expose the wire-level effects that are currently masked.

7.5 Design Recommendations

R1 — Binary serialization.

Replace JSON with MessagePack or raw float32 bytes. Expected reduction: $5\times$ in payload size, $\approx 25\times$ in combined serialization + crypto cost.

R2 — Feature-level offloading.

Partition the pipeline: features locally, classification remotely. Reduces payload from 172 KB to <1 KB; ZTA overhead falls below 1 ms.

R3 — Complexity-adaptive scheduling.

Use the EWMA of local execution time as the scheduling signal. When $\hat{T}_{\text{local}} < L^*$, stay local; when $\hat{T}_{\text{local}} > L^*$, offload. With binary serialization, $L_{\text{ZTA}}^* \approx 3$ ms, making offloading universally beneficial.

R4 — Monitor tail latency, not mean.

Means are often within noise of each other; the offloading benefit materializes at p95/p99 (local 204 ms vs. ZTA 119 ms) where clinical latency constraints are actually tested.

8 Broader Impact: Relevance to Duke University Research and Clinical Settings

EdgeTrust-Offload was developed at Duke University’s Pratt School of Engineering, and its findings have direct relevance to several ongoing research programs and institutional infrastructure initiatives at Duke. We discuss these connections to ground the technical contributions in concrete deployment scenarios.

8.1 Duke Health and HIPAA-Governed Clinical IoT

Duke University Hospital and the Duke Health system operate one of the largest academic medical center networks in the United States. Clinical IoT devices—bedside monitors, wearable ECG/EEG patches, glucose sensors, smart infusion pumps—generate continuous biosignal streams that must be routed, processed, and stored in compliance with HIPAA’s Security Rule, which mandates encryption of all ePHI in transit. As Duke Health expands remote patient monitoring through the DukeHealth@Home program and telestroke/telepsychiatry services, edge compute nodes in nursing units and home-care hubs become natural points for real-time signal processing.

Our results directly answer the question that clinical IoT architects face: *Can we enforce ZTA on every sensor-to-edge-server link without destroying the latency budget?* For normal biosignal workloads (e.g., arrhythmia detection, fall detection), ZTA offloading at ≈ 111 ms exceeds the ~ 20 ms local baseline—but our feature-offload recommendation (R2) projects latency below 25 ms even with full ChaCha20-Poly1305 encryption, meeting most real-time clinical thresholds. For heavy compute workloads (e.g., continuous multi-channel EEG seizure detection with ≈ 130 ms local latency), ZTA offloading at 109 ms already outperforms local execution while guaranteeing p99 latency stability that thermal-throttled embedded hardware cannot provide.

8.2 Duke Biomedical Engineering: Prosthetics and Neural Interfaces

The Duke Department of Biomedical Engineering (BME) is internationally recognized for prosthetics and neural interface research. Groups in the Pratt School develop EMG-controlled prosthetic hands and brain-machine interfaces (BMIs) that require real-time signal decoding at sub-100 ms latency for natural motor control. EMG-based prosthetic control is precisely the application class motivating our BHAM workload [18].

The specific finding that ZTA offloading reduces p95 latency from 204 ms to 119 ms for complex feature hierarchies is directly applicable: prosthetic control systems operating near the 100–150 ms perceptibility threshold [15] cannot tolerate the high tail latency of local ARM processing, but they *can* tolerate a 109–119 ms mean under ZTA if the ZTA stack provides hard latency guarantees. Our crossover framework (L^* analysis) gives prosthetics researchers a principled tool for deciding

whether to add a ZTA-encrypted edge server to a wearable EMG pipeline and, if so, what payload format is required to make it work.

8.3 Duke Smart Home and Campus IoT Infrastructure

The Duke Smart Home program deploys sensor-equipped houses for research into sustainable technology and ambient-assisted living. These deployments mix occupancy sensors, air quality monitors, smart appliances, and health monitoring devices on shared campus network infrastructure. The data generated spans multiple sensitivity levels (anonymous environmental vs. personal health) and multiple regulatory frameworks (FERPA for student residents, HIPAA for health sensors).

Duke’s Office of Information Technology (OIT) has adopted a ZTA posture for all campus networked devices following the 2022 Duke Security Framework revision, requiring Tailscale-or-equivalent VPN overlays for all IoT devices accessing internal services. Our measurements—collected on Duke network infrastructure using Tailscale—provide the first empirical latency budget for this exact deployment architecture. System administrators and researchers deploying sensing nodes on Duke’s network can use Table 1 directly to predict end-to-end latency for a given payload size and workload complexity, and to evaluate whether upgrading to binary serialization (R1) justifies the development effort.

8.4 Duke Robotics and Autonomous Systems Lab

Duke researchers in computer science and electrical & computer engineering develop autonomous robotic systems for inspection, navigation, and human-robot interaction. Field robots increasingly operate in environments requiring ZTA controls (hospitals, industrial facilities, government buildings) where a compute-heavy perception module must offload to a local edge server under encrypted channels.

Our EdgeTrust-Offload framework is directly transferable to robotic perception pipelines: replace the EMG window with a LiDAR scan segment or image patch, replace the gesture CNN with an object detection network, and the same cost model (Eq. 3) and crossover analysis (Eq. 5) apply. The payload-size dimension changes (images are typically 100 KB–5 MB, larger than our 172 KB EMG window), but the crypto overhead model (Eq. 4) scales correctly. For a 1 MB JPEG image payload at 15 MB/s ChaCha20, $E_{\text{crypto}} \approx 133$ ms—already exceeding typical inference time—making binary serialization (R1) non-negotiable for robotic vision under ZTA.

9 Limitations

Single hardware pair. Results are specific to the RPi 3B (ARM Cortex-A53) and Jetson Orin Nano. Faster clients (RPi 4B, Cortex-A72; smartphones, Cortex-A76) have higher ChaCha20 throughput, lowering E_{crypto} and shifting L^* . Results should

be re-measured for each target hardware platform using Eq. (4) with the appropriate R_{ChaCha} .

JSON serialization only. All experiments use Python JSON encoding. Binary formats would reduce both T_{net} and E_{crypto} substantially (Section 6), and may reveal the wire-level congestion sensitivity that is masked by the current serialization floor.

Single workload class. The workload is limited to 8-channel EMG feature extraction and CNN classification. Different compute-to-payload ratios (video frames, LiDAR, audio) will produce different crossover profiles. The cost model (Eqs. 3–5) is general; the constants must be re-measured for each workload.

Synthetic crypto model. E_{crypto} is estimated from R_{ChaCha} rather than directly instrumented inside the WireGuard kernel module. The close agreement between model (22.9 ms) and observed S3–S2 delta (28.3–30.3 ms) validates the model, but the residual 5–7 ms of Tailscale overhead is accounted for only empirically, not by Eq. (4).

No energy measurement. Offloading eliminates local CPU cycles but activates the NIC and Tailscale daemon. A complete trade-off analysis requires power telemetry—critical for battery-powered wearables.

No multi-client contention. All experiments use a single RPi client. GPU inference queue saturation under concurrent requests from multiple nodes is not characterized here.

10 Future Work

Binary payload benchmark. Re-run all four scenarios with MessagePack and raw float32 serialization to isolate wire-level and crypto costs from the current JSON bottleneck. This is the highest-priority follow-up, as it directly validates the $25\times$ reduction projection.

Feature-level offloading implementation. Implement and benchmark the feature-offload architecture: extract the 64-float feature vector locally and offload only that compact representation for GPU classification under full ZTA.

Adaptive weight tuning. Extend the scheduler with on-line Bayesian optimization or a lightweight DQN [4] to learn $\alpha, \beta, \gamma, \delta$ from observed latency distributions, particularly in the vicinity of L^* where small perturbations flip the optimal decision.

Wi-Fi and multi-hop evaluation. Deploy over Wi-Fi 6 (802.11ax) and Tailscale DERP (Internet-relayed) paths to characterize L^* under the higher, more variable RTT typical of real IoT deployments outside controlled LAN environments.

Energy-aware scheduling. Integrate RPi power telemetry (INA219 current sensor) and Jetson tegrastats power readings to add an energy dimension to Eq. (3), enabling Pareto-optimal scheduling between latency and power consumption.

Multi-client GPU contention. Evaluate Jetson GPU queue behavior under simultaneous offload requests from multiple RPi nodes. As Duke Smart Home or Duke Health deployments might have 5–20 concurrent sensing nodes, throughput and fairness under contention are critical for scalability planning.

Hardware crypto acceleration. Test the same pipeline on a device with AES-NI or ARMv8 Cryptography Extension hardware (e.g., RPi 4B with Cortex-A72), where ChaCha20/AES-GCM throughput is $10\times$ higher, projecting $E_{\text{crypto}} \approx 2.3$ ms for the same 172 KB payload and potentially flipping the ZTA offloading decision for normal workloads.

11 Conclusion

This paper presents EdgeTrust-Offload, a real-world measurement study quantifying the latency cost of Zero-Trust security in edge computing task offloading between a Raspberry Pi 3B sensing node and an NVIDIA Jetson Orin Nano edge server. Using a clinically representative EMG gesture recognition workload ($n = 100$ trials per cell, 12 cells total), we characterize the crossover latency L^* under six conditions spanning local execution, insecure LAN offloading, full ZTA offloading, and a three-level synthetic congestion sweep.

Our principal findings are: (1) ChaCha20-Poly1305 encryption of the 172 KB JSON payload costs 23.4 ms on the RPi 3B—77–83% of total ZTA overhead—raising the crossover threshold from ≈ 13 ms to ≈ 88 ms and making local execution always preferable for normal workloads; (2) for compute-intensive workloads, ZTA offloading saves 23 ms mean and reduces p95 latency by $1.7\times$ by eliminating RPi thermal throttle jitter; (3) synthetic congestion up to 60 ms/20 ms/5% loss adds < 1.4 ms to mean latency, revealing that the pipeline is application-layer-bottlenecked, not network-bottlenecked; and (4) switching to binary serialization (R1) or feature-level offloading (R2) would reduce total ZTA offload latency from ~ 111 ms to $\sim 8\sim 26$ ms, making secure offloading universally beneficial.

These results have direct practical implications for Duke Health clinical IoT deployments, Duke BME prosthetics research, and Duke campus IoT infrastructure—and provide a general empirical framework for any edge-AI deployment that must balance latency, security, and computational cost under Zero-Trust constraints. The EdgeTrust-Offload system, benchmark suite, and all raw experimental data are released open-source.

Acknowledgments

The authors thank the Duke University Pratt School of Engineering for computational and hardware resources supporting this project, and Tailscale, Inc., for providing network overlay infrastructure.

AI Usage Disclosure

Generative AI tools (Claude, Gemini Pro) were used to assist with proofreading, grammar refinement, repository scaffolding, and README generation. All experimental design, implementation, data collection, analysis, and written conclusions are the

original work of the authors. No AI tool was used to generate experimental data or interpret results.

References

- [1] M. Satyanarayanan, “The Emergence of Edge Computing,” *Computer*, vol. 50, no. 1, pp. 30–39, Jan. 2017.
- [2] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, “A Survey on Mobile Edge Computing: The Communication Perspective,” *IEEE Commun. Surveys Tuts.*, vol. 19, no. 4, pp. 2322–2358, 2017.
- [3] J. Liu, Y. Mao, J. Zhang, and K. B. Letaief, “Delay-Optimal Computation Task Scheduling for Mobile-Edge Computing Systems,” in *Proc. IEEE ISIT*, 2016, pp. 1451–1455.
- [4] X. Wang, Y. Han, V. C. M. Leung, D. Niyato, X. Yan, and X. Chen, “Convergence of Edge Computing and Deep Learning: A Comprehensive Survey,” *IEEE Commun. Surveys Tuts.*, vol. 22, no. 2, pp. 869–904, 2020.
- [5] X. Chen et al., “Efficient Multi-User Computation Offloading for Mobile-Edge Cloud Computing,” *IEEE/ACM Trans. Netw.*, vol. 24, no. 5, pp. 2795–2808, 2016.
- [6] I. A. Elgendy, W. Zhang, H. He, C. Liu, and A. Kumar, “Efficient and Secure Multi-User Multi-Task Computation Offloading for Mobile-Edge Computing in Multi-Cell Networks,” *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 4, pp. 2410–2422, 2020.
- [7] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, “Zero Trust Architecture,” *NIST Special Publication 800-207*, National Institute of Standards and Technology, Aug. 2020.
- [8] A. Ali, M. Zain ul Abidin Khan, G. Masood, H. T. Mirza, and M. Asim, “Implementing Zero Trust Security with Dual Fuzzy Methodology for Secure Communication and Data Transmission,” *Applied Sciences*, vol. 14, 2024.
- [9] A. Almuseelem, “Secure Latency-Aware Task Offloading Using Federated Learning and Zero Trust in Mobile Edge Computing Environments,” *IEEE Access*, 2025.
- [10] J. A. Donenfeld, “WireGuard: Next Generation Kernel Network Tunnel,” in *Proc. ISOC NDSS*, 2017.
- [11] A. Mackey, P. Bhatt, M. Mahindru, and P. A. Sahu, “A Performance Comparison of WireGuard and OpenVPN,” in *Proc. IEEE COMSNETS*, 2020.
- [12] F. Purba, E. Nasution, and Y. Hardinata, “Securing Data Transmission in IoT Devices Using WireGuard VPN Protocol,” *JOIV: Intl. J. Informatics Visualization*, 2025.
- [13] S. Jeong et al., “TensorRT-Based Framework and Optimization Methodology for Deep Learning Inference on Jetson Boards,” *IEEE Access*, vol. 10, 2022.

- [14] S. Baller, A. Distributed, and M. Herber, “DeepEdgeBench: Benchmarking Deep Neural Networks on Edge Devices,” in *Proc. IEEE IC2E*, 2021.
- [15] M. Zanghieri, S. Benatti, A. Burrello, V. J. Kartsch, F. Conti, and L. Benini, “Robust Real-Time Embedded EMG Recognition Framework Using Temporal Convolutional Networks on a Multicore IoT Processor,” *IEEE Trans. Biomed. Circuits Syst.*, vol. 14, no. 2, pp. 244–256, 2019.
- [16] A. Moin et al., “A Wearable Biosensing System with In-Sensor Adaptive Machine Learning for Hand Gesture Recognition,” *Nature Electronics*, vol. 4, pp. 54–63, 2020.
- [17] K. Kim et al., “Real-Time EMG-Based Hand Gesture Recognition for Edge Devices,” *Sensors*, vol. 23, 2023.
- [18] S. Mick, N. Bhakta, J. Soares, K. Bhakta, and L. J. Hargrove, “BHAm: A Benchmark for Hand Gesture Recognition Using sEMG,” *Scientific Data*, 2024.
- [19] NVIDIA Corporation, “TensorRT Developer Guide, v10,” <https://docs.nvidia.com/deeplearning/tensorrt/>, 2024.
- [20] NVIDIA Corporation, “Jetson Orin Nano Series Module Datasheet,” Rev. 1.0, 2023.
- [21] Tailscale Inc., “How Tailscale Works,” <https://tailscale.com/blog/how-tailscale-works>, 2024.